Zuriel’s Honey Pot Manual

1. Open the Virtual Machine and Log in. Username is Kali and Password is Kali

Cowrie

Firstly we need to enter the directory that contains the Cowrie Honeypot.

“cd cowrie” the path should be “/home/kali/cowrie”

Then cowrie would be able to be booted

“bin/cowrie start”

Splunk

- Firstly we need to start up the Log Visualiser, Splunk to set up the real time alert for both the Brute Force Logs from the Cowrie Honeypot

- Splunk has been set up so that it runs when the honeypot boots up, so open up the web browser and enter this URL to access Splunk

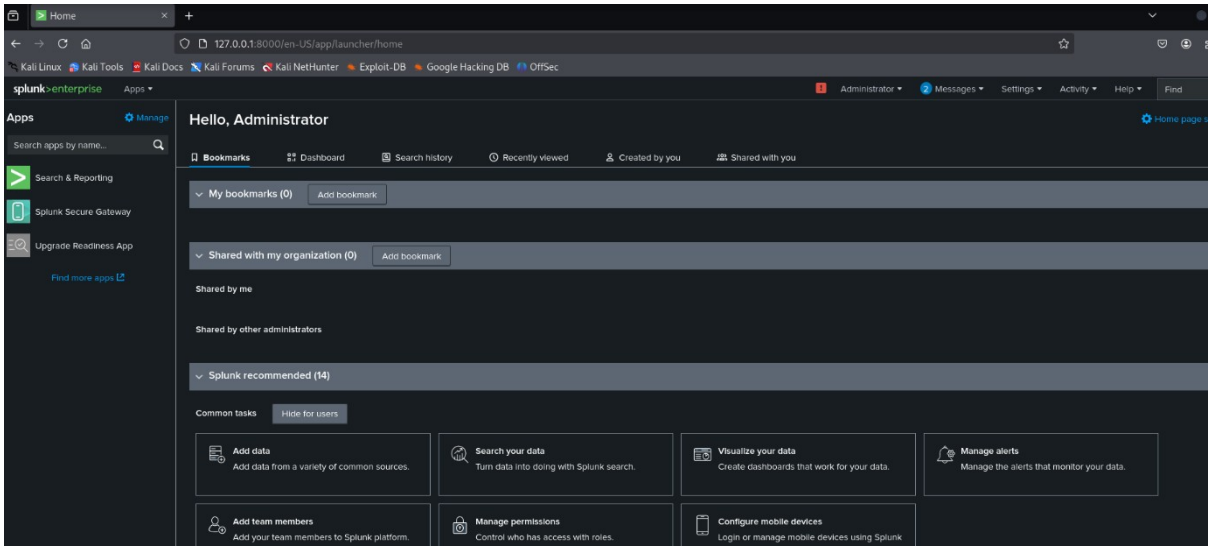
```
┌──(kali㉿kali)-[~/cowrie]
└─$ bin/cowrie start
Using default Python virtual environment "/home/kali/cowrie/cowrie-env"
Starting cowrie: [twistd  --umask=0022 --pidfile=var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie ]...
/home/kali/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:105: CryptographyDeprecationWarning: TripleDES
n cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
/home/kali/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:112: CryptographyDeprecationWarning: TripleDES
n cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
Another twistd server is running, PID 53558

This could either be a previously started instance of your application or a
different application entirely. To start a new one, either run it in some other
directory, or use the --pidfile and --logfile parameters to avoid clashes.
```

http://127.0.0.1:8000

- But if for some reason it hasn't started up run this command in the terminal and put in the URL above into the web browser.

Sudo /opt/splunk/bin/splunk start

Username: "admin"
Password: "admin123"



Towards the left of the screen you'll see Search and Reporting; click that enter this search term.

source="/home/kali/cowrie/var/log/cowrie/cowrie.log" host="kali" sourcetype="Cowrie Log"

This is a basic search filter that tells Splunk to look for logs from the Cowrie honeypot.

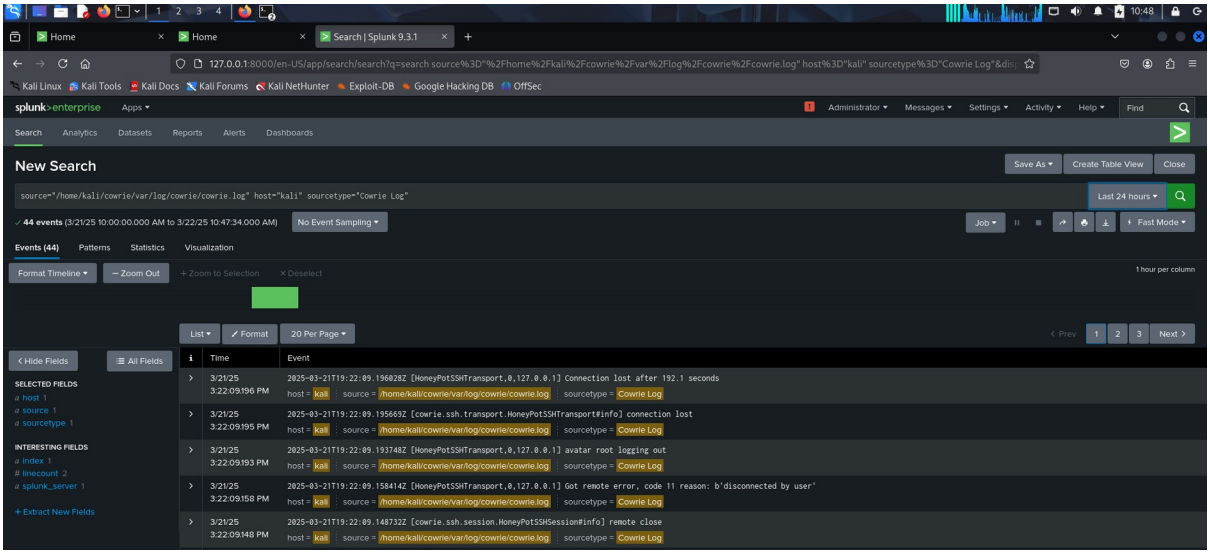
source="/home/kali/cowrie/var/log/cowrie/cowrie.log"

- This tells Splunk to only look at logs from **this specific file** on the machine.
- cowrie.log contains **general honeypot activity**, such as login attempts, file interactions, and attacker behavior.

host="kali"

- The **host field** is the name of the system sending the logs.
- In this case, it's "kali", meaning logs are coming from a machine named "kali" in Splunk.

The outcome is below, showing all the logs of the cowrie logs, displaying all the connections made, the IP addresses that made the connection and the actions the attacker made



The search then displays all the logs of the cowrie logs, displaying all the connections made, the IP addresses that made the connection and the actions the attacker made while connected to the honeypot with changes.

- When your satisfied with everything. Click ‘Save As’ and tap New Dashboard
  Fill out the details and click save to DashBoard

  Instead of manually running searches every time, a dashboard updates automatically to display the latest attack data.
  The dashboard can be set with refresh intervals (e.g., every 30 seconds) to keep an eye on brute-force attempts as they happen.

Brute Force Attack Test

On the attacker VM all the tools have been pre-installed already.

Firstly on the attacker VM, we would need to create a list of possible Usernames and Passwords to run them against the honeypot. This could be done with :

```
echo -e "root\admin\ntest" > usernames.txt
echo -e "root\n1234\admin\password" > passwords.
```

This should return two text files for the corresponding names

To conduct the attack, it would need to be carried out on the command line using Hydra. The command for this below;

```
hydra -L usernames.txt -P passwords.txt ssh:// 192.168.92.132 -s 2222
```

Expected Output:

C. Expected Result

Hydra will attempt combinations and Cowrie , will:

- Log all login attempts
- Record IP address of the attacker
- Log entered commands even if fake

```
┌──(kali㉿kali)-[~]
└─$ hydra -L usernames.txt -P passwords.txt ssh://192.168.92.132 -s 2222

Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
 military or secret service organizations, or for illegal purposes (this is n
on-binding, these ***  ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-04-11 06:
50:18
[WARNING] Many SSH configurations limit the number of parallel tasks, it is r
ecommended to reduce the tasks: use -t 4
[DATA] max 12 tasks per 1 server, overall 12 tasks, 12 login tries (l:3/p:4),
 ~1 try per task
[DATA] attacking ssh://192.168.92.132:2222/
[2222][ssh] host: 192.168.92.132   login: root   password: admin
[2222][ssh] host: 192.168.92.132   login: test   password: password
[2222][ssh] host: 192.168.92.132   login: admin   password: admin
[2222][ssh] host: 192.168.92.132   login: admin   password: password
[2222][ssh] host: 192.168.92.132   login: root   password: 1234
[2222][ssh] host: 192.168.92.132   login: admin   password: root
[2222][ssh] host: 192.168.92.132   login: test   password: root
[2222][ssh] host: 192.168.92.132   login: test   password: 1234
[2222][ssh] host: 192.168.92.132   login: root   password: password
[2222][ssh] host: 192.168.92.132   login: admin   password: 1234
[2222][ssh] host: 192.168.92.132   login: test   password: admin
1 of 1 target successfully completed, 11 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-04-11 06:
50:20
```

With the logs being able to be seen in the dashboard created or at the file location (/home/kali/cowrie/var/log/cowrie/cowrie.log)

Suricata

Open the command Line and enter the command below to start up Suricata

sudo systemctl start suricata

And to confirm its running properly put this command below

sudo systemctl status suricata

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl start suricata


┌──(kali㉿kali)-[~]
└─$ sudo systemctl status suricata

● suricata.service - Suricata IDS/IDP daemon
     Loaded: loaded (/usr/lib/systemd/system/suricata.service; disabled; preset: disabled)
     Active: active (running) since Fri 2025-05-09 17:30:33 EDT; 4s ago
 Invocation: effe455415454b2dae474008701e77b9
       Docs: man:suricata(8)
             man:suricatasc(8)
             https://suricata.io/documentation/
    Process: 8878 ExecStart=/usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricat
   Main PID: 8879 (Suricata-Main)
      Tasks: 1 (limit: 2199)
     Memory: 146M (peak: 146M)
        CPU: 4.987s
     CGroup: /system.slice/suricata.service
             └─8879 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid

May 09 17:30:33 kali systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
May 09 17:30:33 kali suricata[8878]: i: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode
May 09 17:30:33 kali systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
lines 1-18/18 (END)
```

Testing Suricata

The way were going to test Suricata is by generating basic malicious traffic to test the rule detection mechanisms.

Firstly we monitor the log files for any changes with this command

sudo tail -f /var/log/suricata/fast.log

Then open up another Terminal and input this curl command to generate the malicious traffic as we see below

```
┌──(kali㉿kali)-[~]
└─$
curl http://testmynids.org/uid/index.html
uid=0(root) gid=0(root) groups=0(root)

┌──(kali㉿kali)-[~]
└─$
curl http://testmynids.org/uid/index.html
uid=0(root) gid=0(root) groups=0(root)
```

The log file should look something like this

```
┌──(kali㉿kali)-[~]
└─$ sudo tail -f /var/log/suricata/fast.log
05/09/2025-17:41:05.485309  [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Pote
tially Bad Traffic] [Priority: 2] {TCP} 18.245.162.94:80 → 192.168.92.132:51128
05/09/2025-17:41:10.071650  [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Pote
tially Bad Traffic] [Priority: 2] {TCP} 18.245.162.94:80 → 192.168.92.132:37958
```

This shows that it is able to alert and log the traffic that was generated.